

Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
Федерального государственного бюджетного образовательного учреждения высшего
образования
«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»
(МИВлГУ)

Факультет _____ ИТР _____

Кафедра _____ ПИН _____

ЛАБОРАТОРНАЯ РАБОТА №1

по Тестирование программного обеспечения

Тема Модульное тестирование

Руководитель

_____ Колпаков А.А.

(фамилия, инициалы)

_____ (подпись)

_____ (дата)

Студент _____ ПИН-123

(группа)

_____ Тараканов А. Н.

(фамилия, инициалы)

_____ (подпись)

_____ (дата)

Муром 2026

Лабораторная работа №1

Тема: модульное тестирование

Цель работы: освоить базовые навыки применения модульного тестирования с акцентом на выявление дефектов и оценку качества реализации программных компонентов.

Вариант 13

Ход работы:

Задание:

Создание одного класса с 2-4 методами, не требующего внешних зависимостей (баз данных, сети и т.п.), но содержащего логику, подлежащую тестированию.

IpAddressValidator – валидация IPv4-адреса.

Текст промпта: Сгенерируй класс IpAddressValidator на C# с методами:

- IsValidIpv4: проверяет валидность IPv4-адреса, входные параметры: string ipAddress, возвращает: bool.
- GetIpClass: определяет класс IPv4-адреса (A, B, C, D, E) по первому октету, входные параметры: string ipAddress, возвращает: string.

Особые условия: метод GetIpClass должен выбрасывать ArgumentException, если IP-адрес невалиден. Не использовать регулярные выражения, парсить строку вручную или через стандартный IPAddress. Класс должен реализовывать интерфейс IpAddressValidator, находящийся в пространстве имён Lab.Interfaces. Класс должен находиться в пространстве имён Lab.Implementations.GenCode1.

					МИВУ 09.03.04 -13.01			
Изм.	Лист	№ докум.	Подп.	Дата				
Разраб.		Тараканов А. Н.			Модульное тестирование	Лит.	Лист	Листов
Провер.		Колпаков А. А.					2	10
						МИВУ ПИН-123		
Н. контр.								
Утв.								

Листинг GenCode1 IPAddressValidator.cs (Claude):

```
using System;
using System.Net;
using Lab.Interfaces;

namespace Lab.Implementations.GenCode1
{
    public class IPAddressValidator : IIPAddressValidator
    {
        public bool IsValidIpv4(string ipAddress)
        {
            if (string.IsNullOrEmpty(ipAddress))
                return false;

            string[] parts = ipAddress.Split('.');
            if (parts.Length != 4)
                return false;

            foreach (string part in parts)
            {
                if (string.IsNullOrEmpty(part))
                    return false;

                if (!int.TryParse(part, out int octet))
                    return false;

                if (octet < 0 || octet > 255)
                    return false;

                // Запрет ведущих нулей (например, "01")
                if (part.Length > 1 && part[0] == '0')
                    return false;
            }

            return true;
        }

        public string GetIpClass(string ipAddress)
        {
            if (!IsValidIpv4(ipAddress))
                throw new ArgumentException($"IP-адрес '{ipAddress}' не является валидным IPv4-адресом.", nameof(ipAddress));

            string[] parts = ipAddress.Split('.');
            int firstOctet = int.Parse(parts[0]);

            if (firstOctet >= 1 && firstOctet <= 126)
                return "A";
            else if (firstOctet >= 128 && firstOctet <= 191)
                return "B";
            else if (firstOctet >= 192 && firstOctet <= 223)
                return "C";
            else if (firstOctet >= 224 && firstOctet <= 239)
                return "D";
            else if (firstOctet >= 240 && firstOctet <= 255)
                return "E";
            else
                // firstOctet == 0 или 127 (loopback) – не принадлежат
                // стандартным классам
        }
    }
}
```

					МИВУ 09.03.04 -13.01	Лист
						3
Изм.	Лист	№ докум.	Подп.	Дата		

```

        throw new ArgumentException($"Первый октет
'{firstOctet}' не принадлежит ни одному классу IPv4 (A-E).",
        nameof(ipAddress));
    }
}

```

Листинг GenCode2 IPAddressValidator.cs (Grok):

```

using System;
using Lab.Interfaces;

namespace Lab.Implementations.GenCode2
{
    public class IPAddressValidator : IIPAddressValidator
    {
        /// <summary>
        /// Проверяет, является ли строка корректным IPv4-адресом
        /// </summary>
        public bool IsValidIPv4(string ipAddress)
        {
            if (string.IsNullOrEmpty(ipAddress))
                return false;

            string[] parts = ipAddress.Split('.');

            if (parts.Length != 4)
                return false;

            for (int i = 0; i < 4; i++)
            {
                string part = parts[i].Trim();

                // Пустая часть или больше 3 символов → сразу невалидно
                if (string.IsNullOrEmpty(part) || part.Length > 3)
                    return false;

                // Проверка, что все символы — цифры
                foreach (char c in part)
                {
                    if (!char.IsDigit(c))
                        return false;
                }

                // Преобразуем в число и проверяем диапазон
                if (!int.TryParse(part, out int number))
                    return false;

                if (number < 0 || number > 255)
                    return false;

                // Запрещаем ведущие нули (кроме самого числа 0)
                if (part.Length > 1 && part[0] == '0')
                    return false;
            }

            return true;
        }
    }
}

```

					МИВУ 09.03.04 -13.01	Лист
						4
Изм.	Лист	№ докум.	Подп.	Дата		

```

        /// <summary>
        /// Определяет класс IPv4-адреса (А, В, С, D, Е) по первому
октету.
        /// Выбрасывает ArgumentException, если адрес невалиден.
        /// </summary>
        public string GetIpClass(string ipAddress)
        {
            if (!IsValidIpv4(ipAddress))
            {
                throw new ArgumentException("Некорректный формат IPv4-
адреса", nameof(ipAddress));
            }

            string[] parts = ipAddress.Split('.');
            int firstOctet = int.Parse(parts[0]);

            if (firstOctet >= 1 && firstOctet <= 126)
                return "A";

            if (firstOctet >= 128 && firstOctet <= 191)
                return "B";

            if (firstOctet >= 192 && firstOctet <= 223)
                return "C";

            if (firstOctet >= 224 && firstOctet <= 239)
                return "D";

            if (firstOctet >= 240 && firstOctet <= 255)
                return "E";

            // Теоретически сюда дойти невозможно после IsValidIpv4,
            // но оставим для защиты от будущих изменений логики
            throw new ArgumentException("Невозможно определить класс IP-
адреса", nameof(ipAddress));
        }
    }
}

```

Листинг GenCode3 IPAddressValidator.cs (ChatGPT):

```

using System;
using Lab.Interfaces;

namespace Lab.Implementations.GenCode3
{
    public class IPAddressValidator : IIPAddressValidator
    {
        public bool IsValidIpv4(string ipAddress)
        {
            if (string.IsNullOrEmpty(ipAddress))
                return false;

            var parts = ipAddress.Split('.');

            if (parts.Length != 4)
                return false;

            foreach (var part in parts)

```

					МИВУ 09.03.04 -13.01	Лист
						5
Изм.	Лист	№ докум.	Подп.	Дата		

```

        {
            // Пустой октет
            if (string.IsNullOrEmpty(part))
                return false;

            // Лидирующие нули запрещаем (например 01, 001)
            if (part.Length > 1 && part.StartsWith("0"))
                return false;

            if (!int.TryParse(part, out int value))
                return false;

            if (value < 0 || value > 255)
                return false;
        }

        return true;
    }

    public string GetIpClass(string ipAddress)
    {
        if (!IsValidIpv4(ipAddress))
            throw new ArgumentException("Invalid IPv4 address.",
nameof(ipAddress));

        var firstOctet = int.Parse(ipAddress.Split('.')[0]);

        if (firstOctet >= 1 && firstOctet <= 126)
            return "A";

        if (firstOctet >= 128 && firstOctet <= 191)
            return "B";

        if (firstOctet >= 192 && firstOctet <= 223)
            return "C";

        if (firstOctet >= 224 && firstOctet <= 239)
            return "D";

        if (firstOctet >= 240 && firstOctet <= 255)
            return "E";

        // 0 и 127 формально не относятся к классам А-Е
        throw new ArgumentException("IP address does not belong to
classes A-E.", nameof(ipAddress));
    }
}
}

```

Краткий анализ различий:

1. Логика парсинга: все три реализации отказались от регулярных выражений в пользу метода Split('.').
2. Обработка октетов: GenCode 1 и GenCode 3 полагаются на int.TryParse, который по умолчанию игнорирует пробелы. GenCode 2 пошел дальше и

					МИВУ 09.03.04 -13.01	Лист
						6
Изм.	Лист	№ докум.	Подп.	Дата		

добавил ручную проверку каждого символа через `char.IsDigit`, однако допустил логическую ошибку, применив метод `.Trim()`, который принудительно очищает строку от пробелов.

3. Безопасность: все три модели успешно реализовали проверку на null и пустые строки, а также добавили логику запрета ведущих нулей (например, «01»), что критично для корректной валидации IP.

Тесты:

Название теста	Что проверяется	Входные данные	Ожидаемый результат
IsValidIpv4_ValidAddress	Корректный IPv4 адрес	"192.168.1.1"	true
IsValidIpv4_InvalidCharacters	Наличие букв в адресе	"not.an.ip"	false
IsValidIpv4_OctetOutOfRange	Число в октете > 255	"256.0.0.1"	false
GetIpClass_ClassA	Корректность класса A	"10.0.0.1"	"A"
GetIpClass_InvalidIp	Исключение для плохого IP	"999.9.9.9"	ArgumentException
IsValidIpv4_LeadingSpaces	Пробелы в начале/конце	" 192.168.1.1"	false
IsValidIpv4_NullInput	Передача null	null	false
IsValidIpv4_EmptyString	Передача пустой строки	""	false

Листинг UnitTest1.cs:

```
using NUnit.Framework;
using Lab.Interfaces;
using Lab.Implementations.GenCode1;
using System;

namespace Lab;

[TestFixture]
public class IpAddressValidatorTests
{
    private IIpAddressValidator _validator;

    [SetUp]
    public void Setup() { _validator = new IpAddressValidator(); }

    // ТЕСТЫ "ЧЁРНОГО ЯЩИКА"

    [Test]
    public void IsValidIpv4_ValidAddress_ReturnsTrue() {
        Assert.That(_validator.IsValidIpv4("192.168.1.1"), Is.True);
    }

    [Test]
    public void IsValidIpv4_InvalidCharacters_ReturnsFalse() {
        Assert.That(_validator.IsValidIpv4("not.an.ip.address"),
Is.False);
    }

    [Test]
    public void IsValidIpv4_OctetOutOfRange_ReturnsFalse() {
        Assert.That(_validator.IsValidIpv4("256.0.0.1"), Is.False);
    }

    [Test]
    public void GetIpClass_ClassA_ReturnsCorrectClass() {
        Assert.That(_validator.GetIpClass("10.0.0.1"), Is.EqualTo("A"));
    }

    [Test]
    public void GetIpClass_InvalidIp_ThrowsArgumentException() {
        Assert.Throws<ArgumentException>(() =>
_validator.GetIpClass("999.9.9.9"));
    }

    // ТЕСТЫ "БЕЛОГО ЯЩИКА"

    [Test]
    public void IsValidIpv4_LeadingSpaces_ReturnsFalse() {
        Assert.That(_validator.IsValidIpv4(" 192.168.1.1 "), Is.False);
    }

    [Test]
    public void IsValidIpv4_NullInput_ReturnsFalse() {
        Assert.That(_validator.IsValidIpv4(null!), Is.False);
    }

    [Test]
    public void IsValidIpv4_EmptyString_ReturnsFalse() {

```

					МИВУ 09.03.04 -13.01	Лист
						8
Изм.	Лист	№ докум.	Подп.	Дата		


```

    Assert.That(_validator.IsValidIpv4(""), Is.False);
}
}

```

Таблица результатов тестирования:

Тест	GenCode1	GenCode2	GenCode3
IsValidIpv4_ValidAddress	✓ Passed	✓ Passed	✓ Passed
IsValidIpv4_InvalidCharacters	✓ Passed	✓ Passed	✓ Passed
IsValidIpv4_OctetOutOfRange	✓ Passed	✓ Passed	✓ Passed
GetIpClass_ClassA	✓ Passed	✓ Passed	✓ Passed
GetIpClass_InvalidIp (Exception)	✓ Passed	✓ Passed	✓ Passed
IsValidIpv4_LeadingSpaces	✗ Failed	✗ Failed	✗ Failed
IsValidIpv4_NullInput	✓ Passed	✓ Passed	✓ Passed
IsValidIpv4_EmptyString	✓ Passed	✓ Passed	✓ Passed

Анализ причин падения тестов:

Во всех трёх реализациях тест `IsValidIpv4_LeadingSpaces` завершился неудачей (результат `True` вместо ожидаемого `False`).

- В **GenCode 1** и **GenCode 3** это связано с использованием `int.TryParse`, который игнорирует пробелы по краям строки на уровне системной реализации .NET.

- В **GenCode 2** дефект вызван явным вызовом метода `.Trim()`, который удаляет пробелы перед валидацией.

Согласно строгим правилам сетевых протоколов, IP-адрес не может содержать пробелов, однако ни одна нейросеть не реализовала проверку на их отсутствие (например, через проверку длины исходной строки или поиск символа пробела).

Оценка качества реализации:

- GenCode 1 (4/5):** Очень чистая и понятная логика, корректно обработаны ведущие нули и `null`. Минус за стандартную проблему с пробелами.
- GenCode 2 (3.5/5):** Хорошая попытка сделать код с проверкой каждого символа, но использование `.Trim()` — это архитектурная ошибка для строгого валидатора.
- GenCode 3 (4/5):** Современный лаконичный код. Прошел 7 из 8 тестов. Ошибка с пробелами аналогична первой модели.

Как тестирование помогло найти ошибки?

Тестирование методом «чёрного ящика» подтвердило, что базовый функционал работает у всех моделей. Однако применение методов «белого ящика» позволило обнаружить логический изъян, общий для всех моделей — избыточную "гибкость" парсинга чисел, которая позволяет пропускать некорректно отформатированные строки с пробелами.

Как улучшить промпт?

Для повышения качества генерации необходимо формулировать промпт более жестко.

Улучшенная версия: «Сгенерируй строгое решение для валидации IPv4. Запрещено использовать методы, игнорирующие пробелы (типа Trim). Если в любой части строки есть пробел — возвращай false. Обязательно добавь проверку на null».

Вывод: в ходе выполнения работы были освоены базовые навыки применения модульного тестирования с акцентом на выявление дефектов и проведена оценка качества реализации программных компонентов.

					МИВУ 09.03.04 -13.01	Лист
						10
Изм.	Лист	№ докум.	Подп.	Дата		